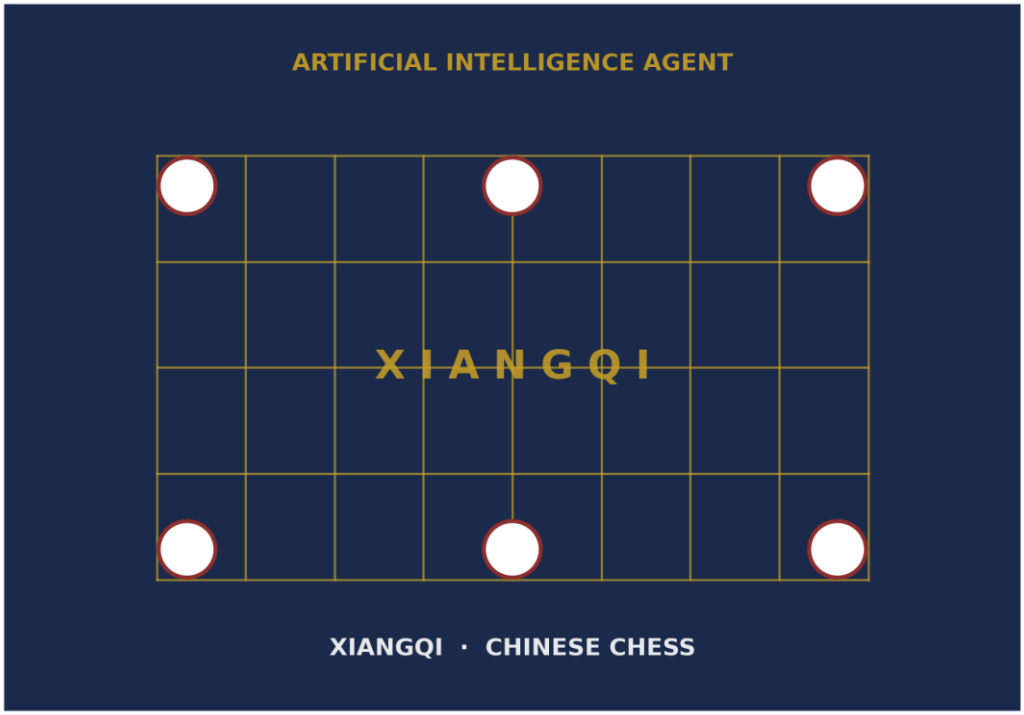
TECHNICAL SPECIFICATION & DEVELOPMENT ROADMAP

HỆ THỐNG AI AGENT CHƠI CỜ TƯỚNG



Project Specification cho Developer & AI Coding Agent

Phạm vi: Xiangqi Environment · Search-based Agents · Deep RL · AlphaGo/AlphaZero-style Agents

Mục lục

Mục lục 2
1. Tổng quan hệ thống 5
Mục tiêu dự án 5
Phạm vi 5
Các thành phần chính 5
Cách các AI Agent tương tác với Game Environment 5
Mục tiêu của việc xây dựng nhiều phương pháp AI khác nhau 6
2. Development Pipeline 7
Dependency giữa các phase 7
Thành phần dùng chung, có thể tái sử dụng 8
3. Phase 1 — Xiangqi Environment 9
Thành phần cần xây dựng 9
Interface đề xuất 9
State Representation 9
4. Các AI Agent 11
Agent 1 — Rule-based / Random / Heuristic 11
A. Mục tiêu 11
B. Kiến thức cần tìm hiểu 11
C. Input 11
D. Core Processing 11
E. Output 11
F. Thành phần cần triển khai 11
G. Pipeline 11
H. Deliverable 11
I. Evaluation 12
5. Agent 2 — Game Tree Search 13
Minimax 13
Alpha-Beta Pruning 13
F. Thành phần cần triển khai 14
H. Deliverable 14
I. Evaluation 14
Agent 3 — Monte Carlo (MCTS) 15
A. Mục tiêu 15
B. Kiến thức cần tìm hiểu 15
C. Input 15
D. Core Processing 15

E. Output....15
F. Thành phần cần triển khai....15
G. Pipeline....15
H. Deliverable....16
I. Evaluation....16
Yêu cầu hỗ trợ từ Environment....16
6. Agent 4 — Neural Network....18
Input encoding....18
Kiến trúc mạng (mức khái niệm)....18
Policy Network....18
Value Network....18
F. Thành phần cần triển khai....18
H. Deliverable....18
I. Evaluation....19
Agent 5 — Reinforcement Learning / Deep Q-Learning (DQN)....20
A. Mục tiêu....20
B. Kiến thức cần tìm hiểu....20
C. Input....20
D. Core Processing....20
E. Output....20
F. Thành phần cần triển khai....20
G. Pipeline....20
H. Deliverable....20
I. Evaluation....21
Đề xuất reward (mức khái niệm)....21
Agent 6 — AlphaGo-style Agent....22
A. Mục tiêu....22
B. Kiến thức cần tìm hiểu....22
C. Input....22
D. Core Processing....22
E. Output....22
F. Thành phần cần triển khai....22
G. Pipeline....22
H. Deliverable....23
I. Evaluation....23
Thành phần tái sử dụng....23
7. Agent 7 — AlphaZero-style Agent....24
Thành phần....24

Vòng lặp chính 24
Khác biệt chính giữa AlphaGo-style và AlphaZero-style 24
Deliverable 25
Evaluation 25
8. Kiến trúc tổng thể 26
Các thành phần dùng chung 26
9. Evaluation & Benchmark 27
Các metric benchmark 27
Định dạng giải đấu 27
Bảng benchmark mẫu 27
10. Development Roadmap 29
11. Project Structure 30
Trách nhiệm các module chính 30
12. Yêu cầu về độ sâu nội dung 31
13. Ghi chú sử dụng tài liệu 32

1. Tổng quan hệ thống

Dự án xây dựng một hệ thống AI Agent chơi Cờ Tướng (Xiangqi), trong đó nhiều phương pháp AI khác nhau — từ random/heuristic đến game-tree search, Monte Carlo, neural network, reinforcement learning và các kiến trúc phong cách AlphaGo/AlphaZero — được triển khai trên cùng một Environment dùng chung.

Mục tiêu dự án

- Xây dựng một Xiangqi Environment chuẩn, có thể tái sử dụng cho toàn bộ agent.
- Triển khai tuần tự nhiều agent với độ phức tạp tăng dần, từ baseline đến các kiến trúc học sâu hiện đại.
- Cung cấp framework benchmark để so sánh khách quan hiệu năng giữa các agent.
- Tạo tài liệu đủ chi tiết để một developer hoặc AI coding agent có thể bắt tay triển khai ngay mà không cần thêm ngữ cảnh.

Phạm vi

Tài liệu bao phủ toàn bộ vòng đời phát triển: từ thiết kế Environment, các agent dựa trên luật/tìm kiếm, đến các agent học sâu và tự chơi (self-play). Không bao gồm triển khai giao diện người dùng đầy đủ hay hạ tầng huấn luyện phân tán quy mô lớn — các nội dung này được nêu ở mức khái niệm khi liên quan.

Các thành phần chính

- Xiangqi Environment — game engine, state representation, legal move generator, rule validation.
- Agent Layer — nhiều loại agent cùng cắm vào Environment qua một interface chung.
- Neural Network & Training Pipeline — dùng chung cho DQN, AlphaGo-style, AlphaZero-style.
- Evaluation & Benchmark Framework — đấu agent với nhau, thu thập số liệu.

Cách các AI Agent tương tác với Game Environment

Mọi agent tuân theo cùng một vòng lặp tương tác: nhận trạng thái hiện tại từ Environment, chọn một hành động hợp lệ, gửi hành động đó cho Environment để thực thi, rồi nhận về trạng thái mới cùng tín hiệu đánh giá/phần thưởng. Việc chuẩn hoá vòng lặp này cho phép các agent khác nhau hoàn toàn về thuật toán nhưng vẫn tương thích với cùng một Environment và cùng một framework benchmark.



Hình 1. Vòng lặp tương tác Agent – Environment.

Mục tiêu của việc xây dựng nhiều phương pháp AI khác nhau

Việc triển khai song song nhiều phương pháp — từ đơn giản đến phức tạp — giúp: (1) có baseline rõ ràng để đo tiến bộ; (2) tách bạch những thành phần nền tảng (Environment, State Representation, Neural Network, MCTS) có thể tái sử dụng xuyên suốt các phase; (3) tạo lộ trình học tập tự nhiên cho đội phát triển, đi từ luật chơi cơ bản đến các kỹ thuật AI hiện đại nhất.

> **LƯU Ý KIẾN TRÚC**
> Environment và Evaluation Framework phải được thiết kế ngay từ Phase 1 để mọi agent phía sau — kể cả AlphaGo-style và AlphaZero-style — có thể tái sử dụng mà không cần viết lại.

2. Development Pipeline

Toàn bộ dự án được chia thành 8 phase phát triển agent cộng với một giai đoạn Evaluation & Benchmark xuyên suốt. Mỗi phase có một trọng tâm kỹ thuật riêng, nhưng phụ thuộc trực tiếp vào các thành phần được xây dựng ở phase trước.



Hình 2. Sơ đồ tổng thể Development Pipeline, từ Environment đến AlphaZero-style Agent.

Dependency giữa các phase

Phase	Phụ thuộc vào
1. Environment	—
2. Rule-based / Random / Heuristic	Environment
3. Game Tree Search (Minimax, Alpha-Beta)	Environment, Evaluation function (Phase 2)

Phase	Phụ thuộc vào
4. Monte Carlo (MCTS)	Environment (clone_state, legal moves, terminal check)
5. Neural Network	State Representation (Phase 1)
6. Reinforcement Learning (DQN)	Environment, Neural Network
7. AlphaGo-style	Neural Network, MCTS, Environment
8. AlphaZero-style	AlphaGo-style, Self-play pipeline
Evaluation & Benchmark	Tất cả các agent

Thành phần dùng chung, có thể tái sử dụng

- Environment — dùng chung tuyệt đối cho mọi agent.
- State Representation — dùng cho search agent, neural network và RL.
- Legal Action Generator — dùng cho mọi agent cần liệt kê nước đi hợp lệ.
- Evaluation Framework (đánh giá bàn cờ) — dùng cho Heuristic, Minimax, Alpha-Beta; là nền tảng ý tưởng cho Value Network.
- Neural Network (Policy + Value) — dùng chung cho DQN (một phần), AlphaGo-style, AlphaZero-style.
- MCTS — dùng chung cho Agent Monte Carlo thuần và cho AlphaGo-style/AlphaZero-style.
- Self-play Pipeline — dùng chung cho AlphaGo-style (tuỳ chọn) và AlphaZero-style (bắt buộc).

3. Phase 1 — Xiangqi Environment

Environment là nền tảng của toàn bộ hệ thống, phải được thiết kế chung cho tất cả các agent ở những phase sau, không được thiết kế riêng cho một thuật toán cụ thể.

Thành phần cần xây dựng

- Board representation — biểu diễn bàn cờ 9x10 và vị trí các quân.
- Piece representation — loại quân, phe, luật di chuyển riêng của từng quân.
- Game state — trạng thái đầy đủ tại một thời điểm (bàn cờ, lượt đi, lịch sử).
- Action/move representation — biểu diễn một nước đi (từ ô nào đến ô nào).
- Turn management — quản lý lượt đi giữa hai bên.
- Legal move generation — sinh danh sách nước đi hợp lệ cho một trạng thái.
- Rule validation — kiểm tra tính hợp lệ của luật (bao gồm luật đặc thù Cờ Tướng như tướng đối mặt).
- Capture handling — xử lý ăn quân.
- Check / checkmate detection — phát hiện chiếu tướng / chiếu bí.
- Game termination — điều kiện kết thúc ván đấu.
- Draw condition — điều kiện hoà (lặp nước, giới hạn số nước không ăn quân...) nếu áp dụng.
- Reset — khởi tạo lại ván đấu.
- Step — thực thi một hành động và cập nhật trạng thái.
- State cloning/copying — sao chép trạng thái nhanh, phục vụ các thuật toán search (Minimax, MCTS).
- Logging / replay — ghi lại lịch sử ván đấu phục vụ debug và tạo training data.

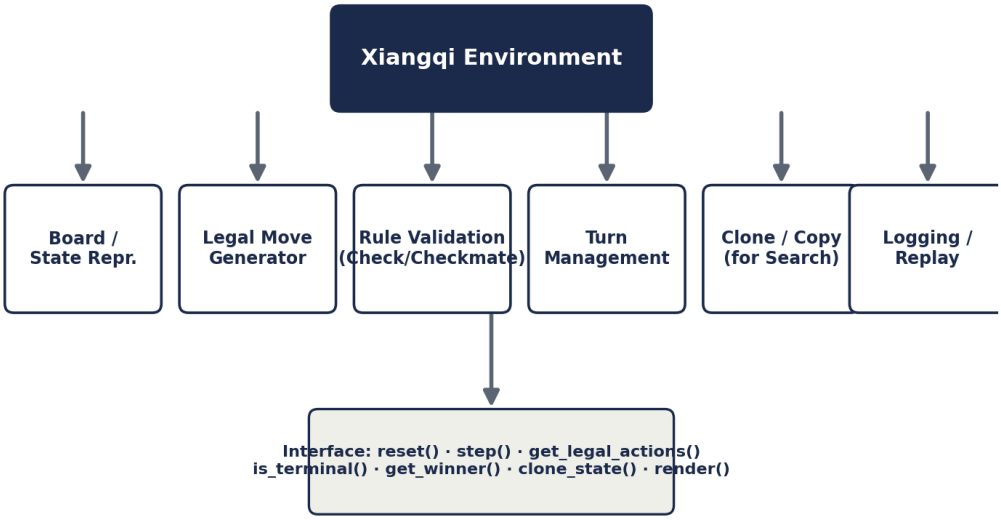
Interface đề xuất

Hàm	Vai trò
reset()	Khởi tạo ván đấu mới, trả về trạng thái ban đầu.
step(action)	Thực thi một hành động, trả về trạng thái mới, reward, trạng thái kết thúc.
get_legal_actions()	Trả về danh sách hành động hợp lệ tại trạng thái hiện tại.
is_terminal()	Kiểm tra ván đấu đã kết thúc hay chưa.
get_winner()	Trả về bên thắng, hoặc hoà, khi ván đấu kết thúc.
clone_state()	Sao chép nhanh trạng thái hiện tại, phục vụ search/simulation.
render()	Hiển thị/tuần tự hoá trạng thái để debug hoặc trực quan hoá.

State Representation

State cần được biểu diễn theo nhiều dạng phù hợp với từng nhóm agent:

- Search Agent (Minimax, Alpha-Beta, MCTS) — biểu diễn dạng ma trận 2D hoặc mảng vị trí quân, tối ưu cho việc sinh nước đi và đánh giá nhanh, cùng với một hàm clone nhẹ để tránh chi phí sao chép lớn khi search sâu.
- Neural Network — biểu diễn dạng tensor nhiều lớp đặc trưng (feature planes): mỗi lớp tương ứng với một loại quân/phe, cộng thêm các lớp phụ trợ như lượt đi hiện tại, số nước đã đi không ăn quân.
- Reinforcement Learning — tái sử dụng tensor của Neural Network, bổ sung thêm thông tin phục vụ tính reward (ví dụ: material trước/sau hành động).



Hình 3. Kiến trúc nội bộ của Xiangqi Environment và các module con.

Mỗi nhóm agent dưới đây được trình bày theo cùng một cấu trúc: Mục tiêu (A) → Kiến thức cần tìm hiểu (B) → Input (C) → Core Processing (D) → Output (E) → Thành phần cần triển khai (F) → Pipeline (G) → Deliverable (H) → Evaluation (I). Cấu trúc nhất quán này giúp developer hoặc AI coding agent dễ dàng đối chiếu giữa các phase.

4. Các AI Agent

Agent 1 — Rule-based / Random / Heuristic

A. Mục tiêu

Thiết lập các baseline đơn giản nhất để kiểm tra tính đúng đắn của Environment và làm mốc so sánh cho mọi agent phức tạp hơn sau này.

B. Kiến thức cần tìm hiểu

- Cách liệt kê và lấy mẫu ngẫu nhiên trong tập nước đi hợp lệ.
- Các rule chiến thuật cơ bản của Cờ Tướng (bảo vệ tướng, không đưa quân vào ô bị ăn miễn phí...).
- Khái niệm hàm đánh giá (evaluation function) tĩnh cho một trạng thái bàn cờ.
- Piece value chuẩn trong Cờ Tướng và các yếu tố ảnh hưởng đến lợi thế vị trí.

C. Input

Trạng thái hiện tại của bàn cờ và danh sách nước đi hợp lệ do Environment cung cấp.

D. Core Processing

Random Agent chọn ngẫu nhiên một nước đi trong danh sách hợp lệ. Rule-based Agent áp dụng một tập rule ưu tiên theo thứ tự cố định (ví dụ: ăn quân có lợi trước, tránh mất quân, bảo vệ tướng). Heuristic Agent tính điểm cho từng nước đi khả dĩ dựa trên hàm đánh giá tổng hợp nhiều yếu tố rồi chọn nước có điểm cao nhất.

E. Output

Một nước đi hợp lệ duy nhất được gửi lại cho Environment thông qua step(action).

F. Thành phần cần triển khai

- Random Agent — lấy legal moves, chọn ngẫu nhiên, dùng làm baseline thấp nhất.
- Rule-based Agent — tập rule ưu tiên theo thứ tự, không cần learning.
- Heuristic Agent — evaluation function dựa trên piece value, material advantage, mobility, king safety, board position.

G. Pipeline



Hình 4. Pipeline xử lý của Heuristic Agent.

H. Deliverable

Ba agent (Random, Rule-based, Heuristic) chạy được đầy đủ ván đấu với Environment, không vi phạm luật, và Heuristic Agent thắng rõ rệt Random Agent trong một loạt trận benchmark.

I. Evaluation

Cho từng agent tự đấu với Random Agent qua nhiều ván (round-robin nhỏ), đo win rate. Heuristic Agent phải có win rate cao hơn đáng kể so với Rule-based và Random.

5. Agent 2 — Game Tree Search

Minimax

Minimax xây dựng một cây trò chơi (game tree) trong đó agent luân phiên đóng vai MAX (tối đa hoá điểm cho bên mình) và MIN (giả định đối thủ tối thiểu hoá điểm của mình). Cây được mở rộng đến một độ sâu (depth) giới hạn, các nút lá được đánh giá bằng evaluation function đã xây dựng ở Agent 1, sau đó điểm số được lan truyền ngược (backpropagate) lên gốc để chọn nước đi tốt nhất.

- Game tree — cấu trúc cây biểu diễn các trạng thái có thể xảy ra.
- MAX player / MIN player — hai vai trò luân phiên theo lượt.
- Depth — độ sâu tìm kiếm, đánh đổi giữa chất lượng quyết định và thời gian tính toán.
- Leaf evaluation — dùng evaluation function tĩnh tại các nút lá khi đạt giới hạn độ sâu.

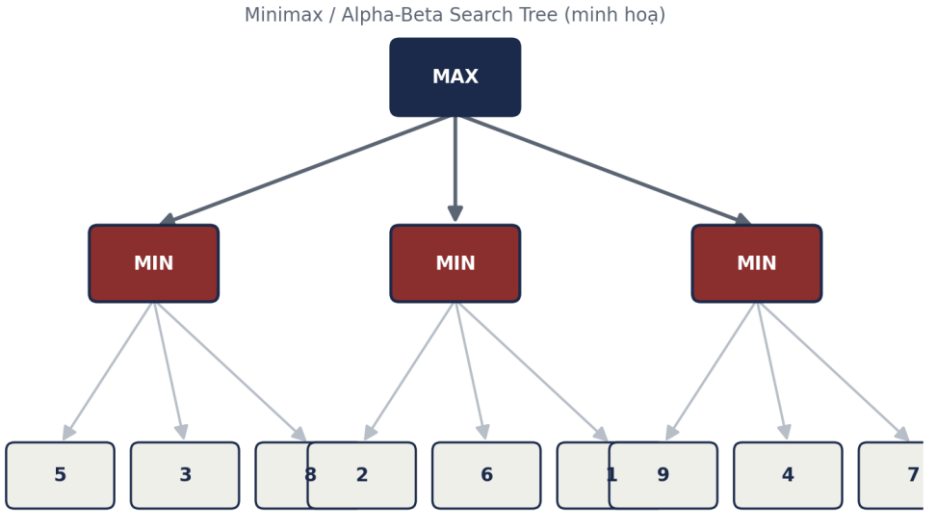


Hình 5. Pipeline xử lý của thuật toán Minimax.

Alpha-Beta Pruning

Alpha-Beta Pruning là một cải tiến của Minimax, cắt bỏ (prune) các nhánh cây chắc chắn không ảnh hưởng đến kết quả cuối cùng, giúp giảm đáng kể số nút cần duyệt mà vẫn cho kết quả giống hệt Minimax thuần. Thuật toán vẫn sử dụng cùng evaluation function đã xây dựng trước đó.

- Alpha — giá trị tốt nhất mà MAX chắc chắn đạt được tính đến hiện tại.
- Beta — giá trị tốt nhất mà MIN chắc chắn đạt được tính đến hiện tại.
- Pruning — dừng khai triển một nhánh khi biết nó không thể cải thiện kết quả.
- Move ordering — sắp xếp thứ tự duyệt nước đi hợp lý giúp tăng hiệu quả cắt tỉa.



Hình 6. Minh hoạ cây tìm kiếm Minimax/Alpha-Beta với các nút MAX, MIN và giá trị lá.

F. Thành phần cần triển khai

- Hàm minimax(state, depth, maximizing_player).
- Hàm alpha_beta(state, depth, alpha, beta, maximizing_player) tái sử dụng evaluation function.
- Cơ chế move ordering đơn giản (ví dụ ưu tiên nước ăn quân trước).

H. Deliverable

Agent Minimax và Alpha-Beta chạy ổn định ở một độ sâu cố định (ví dụ 3–4 ply), Alpha-Beta cho kết quả giống Minimax nhưng số nút duyệt ít hơn rõ rệt.

I. Evaluation

So sánh số nút duyệt (search nodes) và thời gian ra quyết định giữa Minimax và Alpha-Beta ở cùng độ sâu; đấu Alpha-Beta với Heuristic Agent để xác nhận cải thiện win rate.

Agent 3 — Monte Carlo (MCTS)

A. Mục tiêu

Xây dựng agent ra quyết định dựa trên mô phỏng thống kê (Monte Carlo Tree Search) thay vì duyệt cây đầy đủ như Minimax, phù hợp với không gian trạng thái lớn của Cờ Tướng.

B. Kiến thức cần tìm hiểu

- Bốn bước cơ bản của MCTS: Selection, Expansion, Simulation, Backpropagation.
- Khái niệm UCB/UCT (Upper Confidence Bound applied to Trees) để cân bằng giữa khai thác (exploitation) và khám phá (exploration).
- Chi phí và tốc độ của việc mô phỏng ván đấu ngẫu nhiên (rollout).

C. Input

Trạng thái hiện tại của bàn cờ; cây tìm kiếm MCTS được xây dựng và mở rộng dần qua nhiều vòng lặp.

D. Core Processing

Tại mỗi vòng lặp: (1) Selection — đi từ gốc xuống theo công thức UCB/UCT để chọn nút hứa hẹn nhất; (2) Expansion — thêm một hoặc nhiều nút con mới; (3) Simulation — mô phỏng nhanh phần còn lại của ván đấu (rollout ngẫu nhiên hoặc theo policy); (4) Backpropagation — cập nhật thống kê thắng/thua ngược lên các nút đã đi qua. Sau đủ số vòng lặp, nước đi được chọn là nước có số lần thăm (visit count) cao nhất.

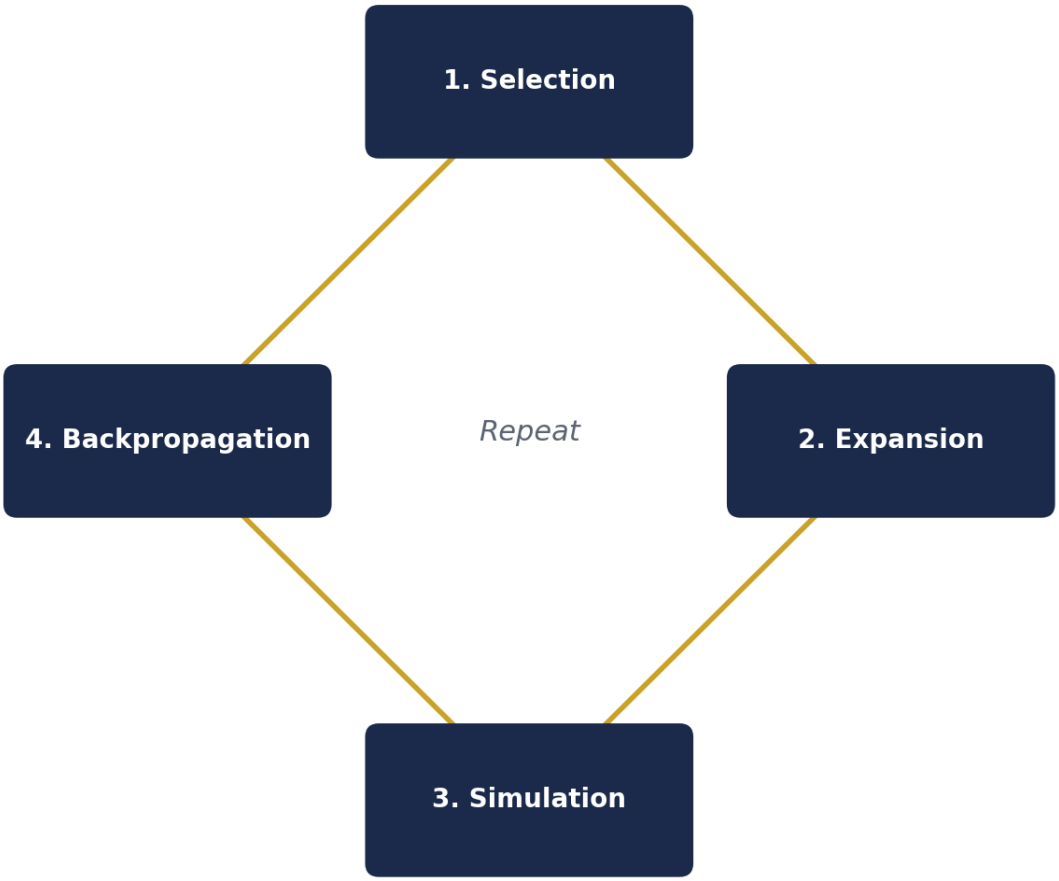
E. Output

Nước đi tốt nhất theo thống kê visit count/giá trị trung bình tại gốc cây MCTS.

F. Thành phần cần triển khai

- Node của cây MCTS: lưu state, số lần thăm, tổng giá trị, danh sách con.
- Hàm UCB/UCT để chọn nút trong bước Selection.
- Cơ chế Expansion tạo nút con từ legal moves.
- Rollout policy cho bước Simulation (đơn giản nhất: random policy).
- Cơ chế Backpropagation cập nhật thống kê dọc theo đường đi.

G. Pipeline



Hình 7. Bốn bước lặp lại của Monte Carlo Tree Search.

H. Deliverable

Agent MCTS chạy được với một ngân sách tính toán cố định (số lượt mô phỏng hoặc thời gian), cho kết quả tốt hơn Random Agent và cạnh tranh được với Heuristic Agent.

I. Evaluation

Đo win rate so với Heuristic Agent và Alpha-Beta ở cùng giới hạn thời gian ra quyết định; theo dõi số lượt mô phỏng (simulations) mỗi nước đi.

Yêu cầu hỗ trợ từ Environment

MCTS đòi hỏi Environment phải hỗ trợ tốt các thao tác sau, nếu không hiệu năng sẽ giảm mạnh do số lượt mô phỏng lớn:

- Fast state cloning — sao chép trạng thái phải đủ nhanh vì được gọi hàng nghìn lần mỗi nước đi.
- Legal move generation — cần chính xác và hiệu quả để không làm chậm bước Expansion.
- Terminal detection — phát hiện kết thúc ván đấu nhanh, phục vụ dừng sớm trong bước Simulation.

6. Agent 4 — Neural Network

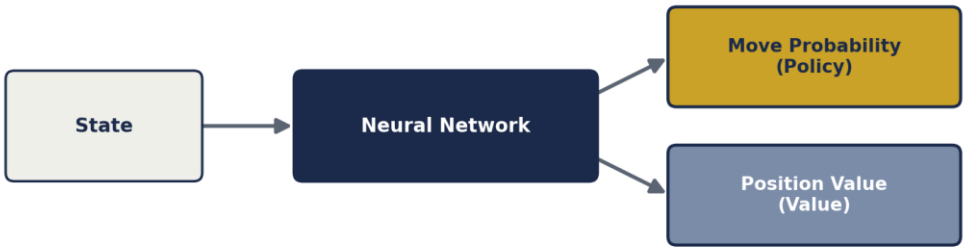
Mục tiêu của phase này không phải là tạo ra một agent hoàn chỉnh, mà là xây dựng nền tảng Neural Network dùng chung cho DQN, AlphaGo-style và AlphaZero-style ở các phase sau.

Input encoding

Trạng thái bàn cờ được mã hoá thành một board tensor gồm nhiều feature planes: mỗi plane là một ma trận nhị phân/số thực đánh dấu vị trí của một loại quân thuộc một phe cụ thể. Có thể bổ sung các plane phụ trợ như lượt đi hiện tại, số nước không ăn quân liên tiếp, hoặc lịch sử vài nước gần nhất.

Kiến trúc mạng (mức khái niệm)

Một backbone tích chập (convolutional) trích xuất đặc trưng không gian từ board tensor, sau đó chia thành hai nhánh đầu ra: một nhánh Policy và một nhánh Value. Không cần thiết kế chi tiết số lớp hay siêu tham số ở giai đoạn đặc tả này; điều quan trọng là chuẩn hoá rõ input/output để các phase sau có thể tái sử dụng trực tiếp.



Hình 8. Kiến trúc khái niệm: State → Neural Network → Policy (xác suất nước đi) và Value (đánh giá thế cờ).

Policy Network

State → Neural Network → Move Probability. Đầu ra là một phân phối xác suất trên không gian hành động, biểu thị agent nên ưu tiên nước đi nào.

Value Network

State → Neural Network → Position Value. Đầu ra là một giá trị vô hướng (thường trong khoảng -1 đến 1) đánh giá mức độ thuận lợi của thế cờ hiện tại đối với bên đang xét.

F. Thành phần cần triển khai

- Module chuyển đổi state sang board tensor (dùng chung với Environment).
- Backbone mạng tích chập trích xuất đặc trưng.
- Đầu ra Policy head và Value head.
- Hàm loss kết hợp (policy loss + value loss) — chuẩn bị cho các phase huấn luyện sau.

H. Deliverable

Một kiến trúc Neural Network hoàn chỉnh nhận board tensor và trả về đồng thời policy distribution và value scalar, sẵn sàng để huấn luyện ở các phase tiếp theo.

I. Evaluation

Kiểm tra shape đầu vào/đầu ra khớp với đặc tả; chạy forward pass thử với dữ liệu ngẫu nhiên để xác nhận không lỗi kỹ thuật trước khi bước sang huấn luyện thực tế.

Agent 5 — Reinforcement Learning / Deep Q-Learning (DQN)

A. Mục tiêu

Huấn luyện một agent học cách chơi thông qua tương tác thử-sai với Environment, sử dụng Neural Network làm bộ xấp xỉ hàm Q-value.

B. Kiến thức cần tìm hiểu

- Các khái niệm nền tảng RL: State, Action, Reward, Episode.
- Q-value và phương trình Bellman ở mức trực quan.
- Đánh đổi Exploration vs Exploitation và chiến lược ε-greedy.
- Replay Buffer và vai trò của việc phá vỡ tương quan giữa các mẫu huấn luyện liên tiếp.
- Target Network và lý do cần một mạng mục tiêu ổn định khi huấn luyện.

C. Input

Trạng thái hiện tại từ Environment, cùng với các transition (state, action, reward, next_state) được lưu trong Replay Buffer.

D. Core Processing

Agent chọn hành động theo chiến lược ε-greedy dựa trên Q-value dự đoán bởi Neural Network. Sau mỗi bước, transition được lưu vào Replay Buffer. Định kỳ, một batch transition được lấy ngẫu nhiên từ buffer để huấn luyện mạng, sử dụng Target Network để tính giá trị mục tiêu ổn định, giảm dao động trong quá trình học.

E. Output

Một hành động được chọn tại mỗi bước, và theo thời gian, một Neural Network Q-value ngày càng chính xác hơn.

F. Thành phần cần triển khai

- Q-Network (tái sử dụng backbone từ Agent 4).
- Target Network — bản sao định kỳ cập nhật của Q-Network.
- Replay Buffer lưu trữ transition.
- Chiến lược ε-greedy với lịch giảm ε theo thời gian.
- Vòng lặp huấn luyện (training loop) cập nhật Q-Network từ batch mẫu.

G. Pipeline



Hình 9. Training loop của Deep Q-Learning: Environment → State → DQN → Action → Reward/Next State → Replay Buffer → Training.

H. Deliverable

Agent DQN huấn luyện ổn định (loss giảm dần, không phân kỳ) và đạt win rate cao hơn Heuristic Agent sau một số lượng episode huấn luyện xác định.

I. Evaluation

Theo dõi đường cong reward trung bình theo episode; đấu định kỳ với Heuristic Agent và Random Agent để đo tiến bộ theo thời gian huấn luyện.

Đề xuất reward (mức khái niệm)

- Win — phần thưởng dương lớn khi thắng ván đấu.
- Loss — phần thưởng âm khi thua ván đấu.
- Draw — phần thưởng trung tính khi hoà.
- Intermediate reward (tuỳ chọn) — phần thưởng nhỏ theo thay đổi material, giúp tín hiệu học dày hơn trong ván đấu dài.

Agent 6 — AlphaGo-style Agent

A. Mục tiêu

Kết hợp Policy Network, Value Network và MCTS thành một agent mạnh, trong đó mạng nơ-ron hướng dẫn tìm kiếm thay vì tìm kiếm mù hoặc rollout ngẫu nhiên thuần tuý.

B. Kiến thức cần tìm hiểu

- Cách Policy Network được dùng để ưu tiên nhánh mở rộng trong MCTS thay vì mở rộng đều.
- Cách Value Network thay thế hoặc bổ trợ cho rollout ngẫu nhiên khi đánh giá nút lá.
- Khái niệm supervised learning từ dữ liệu ván đấu chuyên gia (nếu có), như một cách khởi tạo Policy Network.
- Vai trò của reinforcement learning trong việc tinh chỉnh agent sau bước học có giám sát.

C. Input

Trạng thái bàn cờ hiện tại, được đưa vào cả Policy Network (để định hướng MCTS) và Value Network (để đánh giá vị trí).

D. Core Processing

Policy Network dự đoán phân phối xác suất trên các nước đi khả dĩ, được dùng làm prior để định hướng bước Selection/Expansion trong MCTS thay vì duyệt đều. Value Network cung cấp ước lượng giá trị vị trí, thay thế một phần hoặc toàn bộ cho rollout ngẫu nhiên trong bước Simulation, giúp tìm kiếm hiệu quả hơn nhiều so với MCTS thuần.

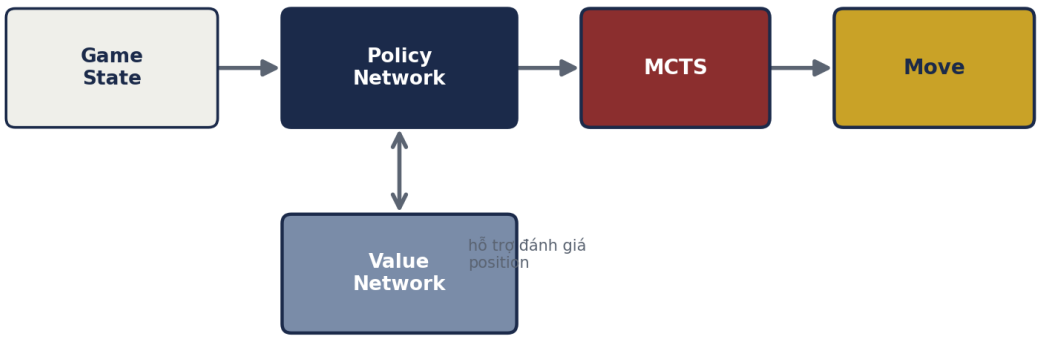
E. Output

Nước đi được chọn từ kết quả tìm kiếm MCTS đã được định hướng bởi Policy và Value Network.

F. Thành phần cần triển khai

- Tích hợp Policy Network làm prior probability cho MCTS.
- Tích hợp Value Network làm hàm đánh giá nút lá, thay thế/bổ trợ rollout.
- (Tuỳ chọn) Bước huấn luyện có giám sát ban đầu từ dữ liệu ván đấu chuyên gia.
- Bước tinh chỉnh bằng reinforcement learning sau huấn luyện có giám sát.

G. Pipeline



Hình 10. Game State → Policy Network → MCTS → Move, với Value Network hỗ trợ đánh giá vị trí.

H. Deliverable

Agent AlphaGo-style đánh bại rõ rệt MCTS thuần và DQN trong benchmark, đồng thời cho thấy hiệu quả tìm kiếm (số simulation cần thiết) tốt hơn MCTS thuần ở cùng chất lượng nước đi.

I. Evaluation

So sánh win rate và số simulation trung bình mỗi nước đi với Agent 3 (MCTS thuần) và Agent 5 (DQN) ở cùng ngân sách thời gian.

Thành phần tái sử dụng

- Neural Network — tái sử dụng trực tiếp kiến trúc và board tensor từ Agent 4.
- MCTS — tái sử dụng khung Selection/Expansion/Simulation/Backpropagation từ Agent 3, chỉ thay đổi cách chọn prior và cách đánh giá nút lá.
- Environment — tái sử dụng nguyên vẹn interface từ Phase 1.

7. Agent 7 — AlphaZero-style Agent

Đây là phase tích hợp cuối cùng, hợp nhất toàn bộ các thành phần đã xây dựng thành một vòng lặp tự cải thiện (self-improving loop), không cần dữ liệu chuyên gia bên ngoài.

Thành phần

- Self-play — agent tự đấu với chính phiên bản mới nhất của mình để sinh dữ liệu huấn luyện.
- MCTS — được định hướng bởi Policy/Value Network hiện tại, giống Agent 6 nhưng dữ liệu huấn luyện đến hoàn toàn từ self-play.
- Policy + Value Network — một mạng duy nhất với hai đầu ra, được huấn luyện lại liên tục.
- Training examples — bộ ba (state, MCTS policy, kết quả ván đấu) thu thập từ self-play.
- Neural Network optimization — huấn luyện lại mạng trên training examples mới.
- Model iteration — đánh giá mạng mới so với mạng cũ, chỉ giữ lại nếu mạng mới tốt hơn.



Hình 11. Vòng lặp tự cải thiện AlphaZero-style: Self-play → Search → Data Generation → Training → Evaluation → New Model.

Vòng lặp chính

Self-play → Search (MCTS định hướng bởi mạng hiện tại) → Data Generation (thu thập training examples) → Training (huấn luyện Policy-Value Network) → Evaluation (so sánh mạng mới với mạng cũ) → New Model (nếu mạng mới thắng, thay thế mạng cũ) → lặp lại chu trình.

Khác biệt chính giữa AlphaGo-style và AlphaZero-style

Tiêu chí	AlphaGo-style	AlphaZero-style
Nguồn dữ liệu huấn luyện ban đầu	Có thể dùng dữ liệu ván đấu chuyên gia (supervised learning)	Hoàn toàn từ self-play, không cần dữ liệu bên ngoài
Số mạng nơ-ron	Có thể tách riêng Policy Network và Value Network	Thường dùng một mạng chung với hai đầu ra
Vòng lặp cải thiện	Huấn luyện có giám sát rồi tinh chỉnh bằng RL	Vòng lặp self-play liên tục, không có bước giám sát ban đầu

Deliverable

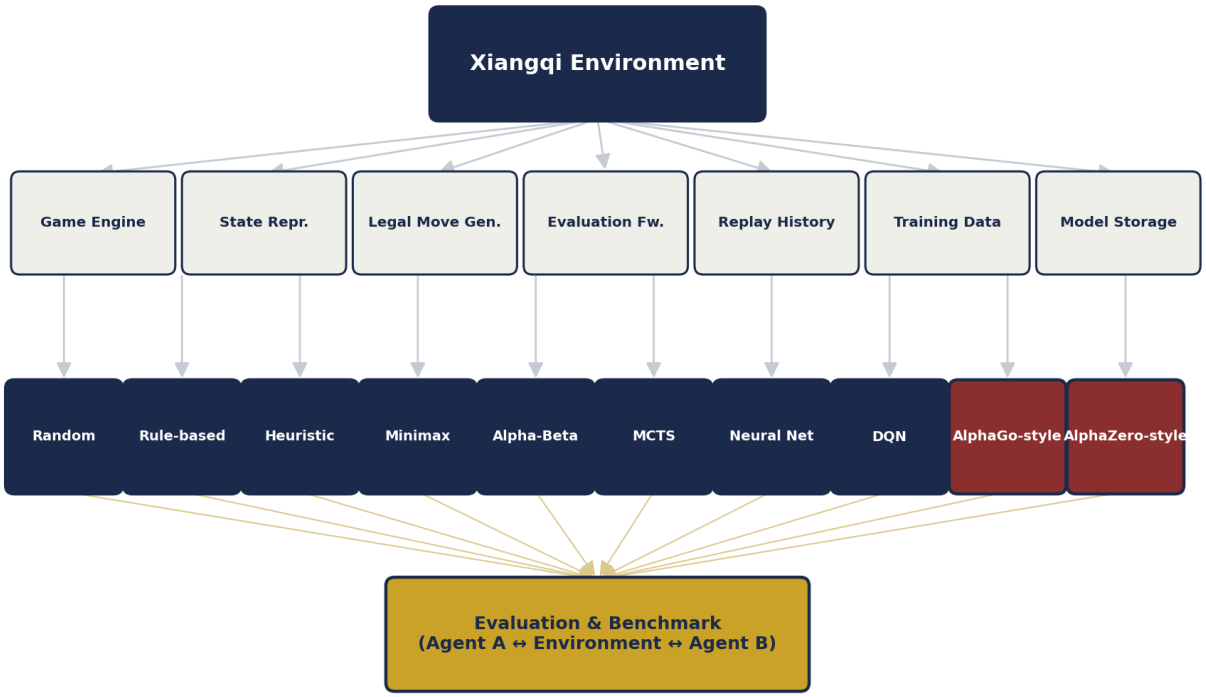
Vòng lặp self-play — training — evaluation chạy được ổn định qua nhiều vòng lặp (iteration), với mạng mới cho thấy xu hướng cải thiện win rate so với mạng ở các iteration trước.

Evaluation

Đấu mạng mới nhất với mạng ở các iteration trước đó và với AlphaGo-style Agent; theo dõi xu hướng win rate qua từng iteration để xác nhận agent đang thực sự cải thiện.

8. Kiến trúc tổng thể

Sơ đồ dưới đây tổng hợp toàn bộ hệ thống: Xiangqi Environment ở trung tâm, các thành phần dùng chung, và toàn bộ 10 agent được xây dựng xuyên suốt các phase.



Hình 12. Kiến trúc tổng thể hệ thống: Environment, các thành phần dùng chung, toàn bộ agent, và tầng Evaluation & Benchmark.

Các thành phần dùng chung

- Game Engine — lõi xử lý luật và trạng thái.
- State Representation — biểu diễn trạng thái dùng chung cho search và neural network.
- Legal Move Generator — sinh nước đi hợp lệ.
- Evaluation Framework — hàm đánh giá tĩnh, nền tảng ý tưởng cho Value Network.
- Replay / Game History — lưu lịch sử ván đấu phục vụ debug và phân tích.
- Training Data — dữ liệu huấn luyện cho DQN, AlphaGo-style, AlphaZero-style.
- Model Storage — lưu trữ các phiên bản mạng nơ-ron qua từng iteration.

9. Evaluation & Benchmark

Framework benchmark cho phép mọi agent đấu với nhau theo cùng một giao thức, đảm bảo so sánh công bằng và có thể lặp lại.



Hình 13. Hai agent tương tác qua Environment trong một trận đấu benchmark (Agent A ↔ Environment ↔ Agent B).

Các metric benchmark

- Win rate — tỉ lệ thắng trên tổng số ván đấu.
- Loss rate — tỉ lệ thua.
- Draw rate — tỉ lệ hoà.
- Average game length — số nước đi trung bình mỗi ván.
- Decision time / move — thời gian ra quyết định trung bình mỗi nước đi.
- Search nodes — số nút duyệt (đối với agent dạng search) mỗi nước đi.
- Training cost — chi phí huấn luyện (nếu áp dụng, cho DQN/AlphaGo-style/AlphaZero-style).

Định dạng giải đấu

Đề xuất tổ chức benchmark theo hình thức round-robin: mỗi agent đấu với mọi agent còn lại một số ván cố định (ví dụ 20–50 ván, đổi phe đi trước/sau đều nhau) để loại bỏ yếu tố may rủi từ lượt đi trước.

Bảng benchmark mẫu

Agent A	Agent B	Win rate A	Win rate B	Draw rate	Avg. game length	Ghi chú
Random	Heuristic					
Heuristic	Alpha-Beta					
Alpha-Beta	MCTS					
MCTS	DQN					
DQN	AlphaGo-style					
AlphaGo-style	AlphaZero-style					

10. Development Roadmap

Phase	Module	Nội dung chính	Dependency	Deliverable
1	Environment	Board, state, luật, legal moves, clone/step/reset	—	Environment chạy đầy đủ, không lỗi luật
2	Rule-based / Random / Heuristic	Baseline agent, evaluation function	Environment	3 agent baseline hoạt động, Heuristic > Random
3	Minimax / Alpha-Beta	Game tree search với evaluation function	Environment, Phase 2	Alpha-Beta nhanh hơn Minimax, cùng kết quả
4	Monte Carlo / MCTS	Selection–Expansion–Simulation–Backprop	Environment (clone nhanh)	MCTS cạnh tranh với Heuristic/Alpha-Beta
5	Neural Network	Board tensor, Policy + Value head	State Representation	Kiến trúc mạng forward pass đúng shape
6	DQN	Q-Network, Replay Buffer, Target Network	Environment, Phase 5	Agent học được, win rate tăng theo thời gian
7	AlphaGo-style	Policy/Value định hướng MCTS	Phase 4, Phase 5	Thắng MCTS thuần và DQN
8	AlphaZero-style	Self-play, training, evaluation loop	Phase 7	Win rate cải thiện qua từng iteration
9	Evaluation & Benchmark	Round-robin, metric, bảng kết quả	Tất cả agent	Bảng benchmark đầy đủ cho toàn hệ thống

11. Project Structure

Cấu trúc source code đề xuất ở mức high-level:

```
xiangqi-ai/ │ ├── environment/          # Game engine, state, luật, legal moves ├── agents/ │
├── random/           # Agent 1 - Random │   ├── heuristic/          # Agent 1 - Rule-based
/ Heuristic │   ├── minimax/           # Agent 2 - Minimax / Alpha-Beta │   ├── mcts/
# Agent 3 - Monte Carlo Tree Search │   ├── neural/             # Agent 4 - Neural Network
(Policy/Value) │   ├── dqn/                # Agent 5 - Deep Q-Learning │   ├── alphago/
# Agent 6 - AlphaGo-style │   └── alphazero/          # Agent 7 - AlphaZero-style │ ├──
models/                 # Lưu trữ phiên bản mạng nơ-ron ├── training/               #
Training loop, self-play pipeline ├── evaluation/             # Benchmark, round-robin,
metric ├── visualization/         # Render bàn cờ, biểu đồ huấn luyện ├── configs/
# Cấu hình siêu tham số theo agent └── tests/                    # Unit test cho Environment
và từng agent
```

Trách nhiệm các module chính

Module	Trách nhiệm
environment/	Chứa toàn bộ logic luật chơi, state, legal move generator — dùng chung cho mọi agent.
agents/	Mỗi thư mục con là một agent độc lập, cùng tuân theo interface chung để cắm vào Environment.
models/	Lưu checkpoint mạng nơ-ron theo từng iteration, phục vụ so sánh và rollback.
training/	Chứa training loop cho DQN và self-play pipeline cho AlphaGo-style/AlphaZero-style.
evaluation/	Chạy round-robin giữa các agent, tính toán và xuất báo cáo benchmark.
visualization/	Công cụ hỗ trợ debug: render bàn cờ, vẽ biểu đồ reward/loss theo thời gian.
configs/	Tập trung toàn bộ siêu tham số, tránh hard-code trong logic agent.
tests/	Unit test đảm bảo Environment và từng agent hoạt động đúng khi refactor.

12. Yêu cầu về độ sâu nội dung

Tài liệu này được xây dựng ở mức Technical Brief / System Specification, không phải một research paper. Mỗi phần ưu tiên trình bày theo trình tự:

What → Why → Input → Processing → Output → Dependency → Deliverable

Tài liệu chủ động không dành nhiều trang cho: lịch sử phát triển thuật toán, chứng minh toán học chi tiết, công thức phức tạp, hay các phần giới thiệu AI chung chung không gắn với dự án cụ thể. Mỗi thuật toán chỉ được giải thích ở mức đủ để developer trả lời được câu hỏi trọng tâm: “Cần tìm hiểu gì và cần xây dựng gì tiếp theo?”

13. Ghi chú sử dụng tài liệu

Tài liệu này được thiết kế để dùng trực tiếp làm project brief/context cho developer hoặc AI coding agent khi bắt đầu triển khai dự án. Toàn bộ roadmap, dependency giữa các phase, input/output của từng agent, và các thành phần tái sử dụng đã được trình bày rõ ràng ở các phần trước, kèm sơ đồ trực quan cho từng giai đoạn chính.

> **LƯU Ý TRIỂN KHAI**
>
> Trình tự triển khai bắt buộc phải bắt đầu từ Environment (Phase 1). Mọi agent — kể cả AlphaGo-style và AlphaZero-style — đều phụ thuộc trực tiếp hoặc gián tiếp vào chất lượng của Environment và State Representation được xây dựng ở phase này.